

File: mL_algo_from_scratch/examples/run_examples.py

Purpose: Linear Regression implementation for continuous value prediction

```
"""Small demo script showing univariate, bivariate, multivariate linear regression, KNN and Naive Bayes usage.
This file uses simple variable names and prints so it's easy to follow.
"""
import numpy as np
```

=== Imports ===

```
from mL_algo_from_scratch.linear_regression import LinearRegression
from mL_algo_from_scratch.knn import KNN
from mL_algo_from_scratch.naive_bayes import GaussianNB
```

=== Functions ===

Logic Focus: Defined task specifically for 'linreg_univariate_demo'

```
def linreg_univariate_demo():
    # y = 2*x + 1 + noise
    rng = np.random.RandomState(0)
    X = rng.rand(100) * 10
    y = 2.0 * X + 1.0 + rng.randn(100) * 1.0

    model = LinearRegression()
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
    model.fit_closed_form(X, y)
    print("Univariate closed-form theta:", model.theta)
    print("MSE:", model.score(X, y))
```

Math Alert: This involves calculus! Derivatives are used to update weights.

```
    # gradient descent
    model2 = LinearRegression()
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
    model2.fit_gradient_descent(X, y, lr=0.001, epochs=20000)
    print("Univariate GD theta:", model2.theta)
    print("MSE GD:", model2.score(X, y))
```

Logic Focus: Defined task specifically for 'linreg_bivariate_demo'

```
def linreg_bivariate_demo():
    rng = np.random.RandomState(1)
    x1 = rng.rand(200) * 5
    x2 = rng.rand(200) * 3
    y = 3.0 * x1 - 2.0 * x2 + 0.5 + rng.randn(200) * 0.5
    X = np.vstack([x1, x2]).T
    model = LinearRegression()
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
    model.fit_closed_form(X, y)
    print("Bivariate theta:", model.theta)
    print("MSE:", model.score(X, y))
```

Logic Focus: Defined task specifically for 'linreg_multivariate_demo'

```
def linreg_multivariate_demo():
    rng = np.random.RandomState(2)
    n = 300
    d = 5
    X = rng.randn(n, d)
    true_theta = np.arange(1, d + 1) * 0.5
    y = X @ true_theta + 0.3 + rng.randn(n) * 0.3
```

```
model = LinearRegression()
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
model.fit_closed_form(X, y)
print("Multivariate theta (first elements):", model.theta[:6])
print("MSE:", model.score(X, y))
```

Logic Focus: Defined task specifically for 'knn_demo'

```
def knn_demo():
    rng = np.random.RandomState(3)
    # binary classification on 2D blobs
    X0 = rng.randn(50, 2) + np.array([0, 0])
    X1 = rng.randn(50, 2) + np.array([3, 3])
    X = np.vstack([X0, X1])
    y = np.array([0] * 50 + [1] * 50)
    model = KNN(k=5, task="classification")
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
model.fit(X, y)
acc = model.score(X, y)
print("KNN training accuracy (k=5):", acc)
```

Logic Focus: Defined task specifically for 'naive_bayes_demo'

```
def naive_bayes_demo():
    rng = np.random.RandomState(4)
    X0 = rng.randn(100, 2) + np.array([-1, 0])
    X1 = rng.randn(100, 2) + np.array([2, 1])
    X = np.vstack([X0, X1])
    y = np.array([0] * 100 + [1] * 100)
    model = GaussianNB()
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
model.fit(X, y)
print("Naive Bayes training accuracy:", model.score(X, y))
```

=== Execution Block (Main) ===

```
if __name__ == "__main__":
    print("--- Linear Regression: univariate ---")
    linreg_univariate_demo()
    print("--- Linear Regression: bivariate ---")
    linreg_bivariate_demo()
    print("--- Linear Regression: multivariate ---")
    linreg_multivariate_demo()
    print("--- KNN demo ---")
    knn_demo()
    print("--- Naive Bayes demo ---")
    naive_bayes_demo()
```